

Scenaric — Backend Build Plan

A step-by-step spec for building the AI backend behind Scenaric's nine-tile methodology tracker — an AI-native implementation of Peter Schwartz's scenario-planning process ("The Art of the Long View"). Read alongside the existing prototype in `app.jsx`, `data.js`, and `components/*.jsx`.

0. Principles

- 1. **Multi-project is foundational, not bolted on.** Every resource (sources, signals, matrix, scenarios, storyline, indicators, strategies) is scoped to a `project_id`. A user/org owns many projects; the prototype's `projects[]` array + `activeProjectId` is the UI contract — the backend must mirror it exactly (see §1).
- 2. **Grounding over generation.** Every AI step reads from the project's own knowledge base and prior steps — never general world knowledge presented as project fact. If evidence is thin, the model must say so (a confidence field and an explicit "insufficient evidence" path), not fabricate detail. This mirrors the Matrix's real "Independence unclear" state and the Storyline's real "gaps in chain" state — the AI is allowed, and expected, to surface uncertainty rather than fill it in.
- 3. **Structured output only.** Every AI call returns strict JSON matching a schema (via tool-calling / JSON mode), never freeform prose to parse. This is the primary anti-hallucination lever: schemas force the model to cite the source ids it used and to emit null/low-confidence fields instead of inventing values.
- 4. **Schwartz's method is the spine, not a vibe.** The 9 tiles are a literal implementation of Schwartz's 8-step process (see mapping below) plus one product-level extension (Strategic Options). Every prompt below encodes the actual definition of that step from the book — not a generic "brainstorm ideas" prompt.
- 5. **Deterministic where it should be.** Classification/scoring calls (impact, uncertainty, orthogonality) run at low temperature with fixed rubrics. Generative calls (narrative prose) run at moderate temperature but are still constrained to cite the causal chain/signals they're built from.

1. Schwartz's 8 steps → Scenaric's 9 tiles → pages

Schwartz's book defines 8 steps. Scenaric splits "Key Forces" and "Driving Forces" into two tiles for the product's Knowledge Base → Signals Library flow, and adds a 9th tile, Strategic Options, as the "so what" layer the book leaves implicit.

#	SCHWARTZ STEP (BOOK)	SCENARIC TILE	PAGE / COMPONENT
1	Identify the focal issue or decision	Focal question	Onboarding, Settings → Project

2	Key forces in the local (task) environment	Key forces	page-knowledge.jsx
3	Driving forces — macro-environmental (STEEP)	Driving forces	page-signals.jsx
4	Rank by importance & uncertainty	Rank forces	page-matrix.jsx
5	Select scenario logics (the 2x2 skeleton)	Scenario logics	build-scenarios-modal.jsx, reaxis-modal.jsx
6	Flesh out the scenarios	Narratives	page-narrative.jsx, page-storyline.jsx
7	Implications	Implications	page-narrative.jsx (Implications block)
8	Selection of leading indicators / signposts	Indicators	page-monitoring-settings.jsx
+	<i>(not in the book — product extension)</i>	Strategic options	page-strategy.jsx

2. Data model (multi-project)

One Postgres-style schema, every table scoped by `project_id` except `orgs`, `users`, and `projects` itself. This is a direct backend for the existing `usePersistentState` keys in `app.jsx` — replace `localStorage` with these tables and the frontend contract doesn't change.

TABLE	KEY COLUMNS	NOTES
orgs	id, name, plan	Billing boundary
users	id, org_id, name, email, role	
projects	id, org_id, name, focal_question, refined_focal_question, horizon, industry, summary, steps_complete, archived, created_by	Mirrors data.js: projects[]. steps_complete is server-derived, not client-set (§6).
sources	id, project_id, name, type, status, storage_url, extracted_text, uploaded_by	Knowledge Base raw uploads (docs/audio/survey/web)
interviews	id, project_id, source_id, participant_name, role, transcript, tag(STEEP), key_quote	
insights	id, project_id, source_id, text, category(STEEP), confidence	AI-extracted from sources; feeds Key Forces
signals	id, project_id, category(STEEP), source, title, body, impact(1-5), uncertainty(Low/Med/High), origin(ai/user/insight_id), status	Driving Forces library
matrix_dots	id, project_id, signal_id, x, y, is_critical_axis, bucket(critical_uncertainty/predetermined/background/wildcard), bucket_rationale	Rank Forces placement; x/y persisted 0–100. bucket is AI-classified and persisted, not recomputed client-side — Step 5/6 read predetermined and wildcard signals directly from here.
axes	id, project_id, x_signal_id, y_signal_id, x_label, y_label, independence_state, independence_rationale	Exactly 2 rows max active per project (Schwartz's 2x2 constraint)
scenarios	id, project_id, axes_id, quadrant(TL/TR/BL/BR), name, tagline, summary, narrative, color, is_archived	Always 4 live per active axes set
storyline_nodes	id, project_id, scenario_id, phase, category, title, body, year, strength, signal_id(nullable)	Causal chain cards
storyline_edges	id, project_id, scenario_id, from_node_id, to_node_id, relationship, confidence	
implications	id, project_id, scenario_id, text, category(capital/hiring/tech/partners/other)	
indicators	id, project_id, scenario_id, name, status(On track/Watch/Alert), trend, note, last_checked	Signposts
strategies	id, project_id, name, notes, risk, cost, robust_in(scenario_id[])	

ai_runs	id, project_id, step, prompt_version, input_hash, output_json, model, confidence, created_at	Audit log — every generation call, replayable and reviewable
---------	--	--

Project switch contract: the client sends `activeProjectId` on every request (header or path param, e.g. `/api/projects/:id/signals`). No endpoint should ever mix data across projects — this is the #1 place a shared vector index or a global RAG context leaks cross-project.

3. Global anti-hallucination scaffold

Reused by every prompt below, not repeated in each section.

SYSTEM PROMPT PREAMBLE (PREPEND TO ALL CALLS)

You are Scenaric's scenario-planning analyst. You work **strictly** from the project data provided in this request — the knowledge base, signals, axes, and prior steps. You are implementing Peter Schwartz's scenario-planning method from "The Art of the Long View." Follow the definition of the current step exactly; do not substitute a generic brainstorming approach.

Rules (violating any of these is a failure):

1. Never invent a fact, statistic, date, or named entity that is not present in the supplied context. If you need a plausible illustrative detail, you may generalize from a cited signal but must not state it as if it were sourced when it is inferred — mark inferred content in the ``inference`` field of your output, never inline in prose.
2. Every claim you output must be traceable to one or more input ids (`signal_id` / `source_id` / `node_id`). Populate ``grounded_in: [...]`` with those ids. If a claim has no grounding, do not include it.
3. If the input evidence is insufficient to complete the step confidently, return ``sufficient_evidence: false`` and a ``gap`` description instead of guessing. Do not pad with generic filler to look complete.
4. Output **ONLY** the JSON matching the provided schema. No prose outside the schema, no markdown, no preamble.
5. Never break the 2-axis / 4-quadrant structure, the 5-phase storyline structure, or the STEEP taxonomy (Social, Technological, Economic, Ecological, Political) — these are fixed by the method, not creative choices.

Enforcement: pass a JSON Schema via the model's structured-output / tool-calling mode (not "please respond in JSON") so malformed output is rejected at the API layer before it reaches the DB. Reject and retry once on schema validation failure; on second failure, surface `sufficient_evidence: false` to the UI rather than showing a hallucinated result.

4. Step 1 — Focal question (onboarding.jsx)

Schwartz: the focal issue must be a real pending decision, specific enough to test scenarios against, with an explicit time horizon.

ENDPOINT	TRIGGER
POST /projects/:id/focal-question/refine	"Refine with AI" button, onboarding step 1

PROMPT (TASK-SPECIFIC SUFFIX)

Task: Sharpen the user's rough focal question into Schwartz's required form: a single decision-oriented question, bounded by the stated time horizon, specific about WHO is deciding and WHAT the decision changes.

Input: { raw_question: string, horizon: string, industry: string }

Requirements for the refined question:

- Must be answerable "yes we should" / "no we shouldn't" or as a choice among named options – not an open-ended topic ("the future of X").
- Must name the actual decision-maker's scope (the company / product / market), not a generic industry question.
- Must include the time horizon verbatim.
- Must NOT introduce new facts (competitors, markets, financial figures) that were not in raw_question – reword and sharpen only.

Output schema:

```
{ refined_question: string, changed: boolean, rationale: string[] /* 1-3 short bullets on what was sharpened and why */ }
```

If raw_question is already well-formed, set changed:false and return it unmodified rather than rewriting for the sake of it.

5. Step 2 — Key forces (page-knowledge.jsx)

Schwartz: forces in the immediate/local environment — customers, suppliers, competitors, stakeholders — distinct from macro driving forces.

ALL FOUR SOURCE TYPES CONVERGE ON ONE INSIGHT SHAPE

Docs, Audio, Survey, and Web are ingestion paths only — every one of them normalizes into the same insights row before extraction, and all land in a single "AI Extracted Insights" feed (filterable by source type, not four separate tabs):

SOURCE TYPE	PRE-STEP BEFORE EXTRACTION	QUOTE FIELD HOLDS
Doc (PDF/DOCX)	text extraction + chunking	verbatim sentence from the text
Audio	transcription (speaker-attributed)	verbatim transcript line + speaker name/role
Survey (CSV)	aggregation across responses	quantified statement, e.g. "78% of respondents selected X"
Web	page scrape → text	verbatim scraped sentence + source URL

ENDPOINT	TRIGGER
POST /sources/:id/extract	File finishes upload/transcription/scrape/aggregation — same endpoint for all 4 types, dispatches on source.type
POST /projects/:id/insights/extract-all	"Extract insights" button

PROMPT

Task: Extract key local-environment forces from ONE source document/transcript. Key forces = stakeholders and actors that directly interact with the focal decision (customers, competitors, regulators the org negotiates with directly, suppliers, partners, internal capability constraints) – NOT macro trends (those are Step 3, driving forces).

Input: { project_focal_question: string, source_id: string, source_text: string /* full extracted text, chunked if long */ }

Extraction rules:

- Only extract statements actually present in source_text. Quote or tightly paraphrase – do not synthesize claims the source doesn't make.
- Each insight must include a verbatim `quote` (<= 40 words) copied from source_text as evidence. If you cannot produce a real quote, drop the insight.
- Classify each insight's STEEP category ONLY if it's a driving force; key forces (this step) are tagged `local\_actor` instead.
- Cap output at the 8 highest-signal insights per source. Do not pad to hit a round number.

Output schema:

```
{ sufficient_evidence: boolean,
  insights: [{ text: string, quote: string, actor_type: string,
               confidence: "high"|"medium"|"low" }] }
```

6. Step 3 — Driving forces (page-signals.jsx)

Schwartz: macro-environmental forces across Social, Technological, Economic, Ecological, Political (STEEP) categories that shape the focal question but sit outside the org's direct control.

Insights → Signals is not 1:1. Insights are local/task-environment evidence tied to one source. Signals are synthesized macro uncertainties that may aggregate several insights across multiple sources and source types (a PDF clause + an interview quote + a survey stat can all ground the same signal). Not every insight graduates — only ones that reveal a macro-environmental uncertainty do; purely local-actor insights (e.g. an interviewee's staffing complaint) stay in Key Forces and ground later Implications/Narrative directly, never entering the Matrix.

ENDPOINT	TRIGGER
POST /projects/:id/signals/suggest	"Add Signal" → AI suggestion tab, or auto-run after insight extraction
POST /insights/:id/merge-into-signal	" + Merge into Signal" on an insight card (Knowledge Base)
POST /projects/:id/signals/score	Batch impact/uncertainty scoring for signals lacking scores

PROMPT — MERGE INSIGHT INTO SIGNAL (USED BY " + MERGE INTO SIGNAL")

```
Task: Given ONE insight the user wants to promote from Key Forces into Driving Forces, decide whether it supports an EXISTING signal or requires a NEW one. Never silently duplicate a signal that already covers this uncertainty.

Input: { insight: {id, text, quote, actor_type, source_type},
        existing_signals: [{id, title, category, body}] }

Decision procedure:
1. If insight's underlying macro uncertainty semantically matches an existing_signals entry (same driver, not just same keyword), return action:"attach" with that signal_id — the insight becomes additional grounded_in evidence, no new signal is created.
2. Else if insight does not describe a macro-environmental force at all (it's purely local-actor: a specific customer/competitor/internal fact with no STEEP-level trend behind it), return action:"reject" with a reason — it stays a Key Force only, it must not become a signal.
3. Else return action:"create_new" with a drafted signal (title, body, category) grounded ONLY in this insight's text/quote — no invented detail beyond it.

Output schema:
{ action: "attach"|"reject"|"create_new",
  signal_id?: string, reason?: string,
  new_signal?: { title, body, category } }
```

PROMPT — GENERATE CANDIDATE SIGNALS FROM INSIGHTS

Task: Convert local insights + the focal question into STEEP-classified driving-force signals, each representing ONE distinct macro uncertainty the scenario set must consider.

Input: { focal_question: string, horizon: string, industry: string, insights: [{id, text, actor_type}], existing_signals: [{id,title,category}] }

Rules:

- Every new signal must be grounded_in ≥ 1 insight id, OR explicitly flagged `origin:"external\_pattern"` if it names a well-established macro trend for this industry/horizon not present in insights (allowed, but must be flagged, never presented as sourced from the user's data).
- Do NOT propose a signal that duplicates an existing_signals title or underlying uncertainty – check semantic overlap, not just exact text.
- Each signal needs exactly one STEEP category: Social, Technology, Economic, Ecological, or Political.
- Write `title` as a factual trend statement (≤ 8 words), `body` as a 2-sentence explanation of the mechanism connecting it to the focal question, and cite `source` as the real originating source name (e.g. "Internal interview – Jan Oosterom") – never a fabricated publication name.
- Cap at 6 new signals per call.

Output schema:

```
{ signals: [{ title, body, category, source, grounded_in: string[],  
              origin: "insight"|"external_pattern" }] }
```

PROMPT — IMPACT / UNCERTAINTY SCORING (DETERMINISTIC, TEMP 0)

Task: Score ONE signal on Schwartz's two axes used for the ranking matrix: IMPACT (how much would this reshape the focal decision if it occurred) and UNCERTAINTY (how unpredictable is its outcome/direction).

Input: { focal_question, signal: {title, body, category} }

Rubric (apply literally, do not freelance a different scale):

Impact 1 = negligible effect on the decision; 5 = would overturn the strategy entirely.

Uncertainty Low = outcome is largely predetermined/trending;

Medium = plausible range of outcomes, some predictability;

High = genuinely unknowable direction today.

Output schema:

```
{ impact: 1|2|3|4|5, uncertainty: "Low"|"Medium"|"High",  
  rationale: string /* one sentence, cites the mechanism, not a vibe */ }
```


7. Step 4 — Rank forces (page-matrix.jsx)

Schwartz: plot every force on importance x uncertainty; the two most important AND most uncertain become the scenario axes. Everything else sorts into Predetermined / Background / Wildcard — wildcards (low-probability, high-impact discrete shocks) are a separate category, not just another point on the continuous grid.

ENDPOINT	TRIGGER
POST /projects/:id/matrix/classify	Matrix page load / after any new signal is scored — batch-assigns bucket to every unbucketed signal and persists it to matrix_dots
GET /projects/:id/matrix/buckets	Matrix page load — returns signals grouped by bucket (critical_uncertainty list feeds axis picker; predetermined/background/wildcard render as separate rails)
POST /projects/:id/matrix/independence-check	User selects 2 axis signals from the critical_uncertainty list (mirrors existing assessIndependence heuristic in page-matrix.jsx, now AI-backed instead of a pseudo-random hash)

PROMPT — BUCKET CLASSIFICATION (DETERMINISTIC, TEMP 0)

```
Task: Assign ONE signal to exactly one of Schwartz's four rank-forces buckets, given its already-scored impact and uncertainty.

Input: { signal: {title, body, category, impact(1-5), uncertainty(Low/Med/High)},
        is_discrete_shock_candidate: boolean /* heuristic pre-check: title/body describes a sudden, low-probability, high-consequence event rather than a continuous trend */ }

Decision procedure (apply literally, in order):
1. If is_discrete_shock_candidate is true AND impact >= 4, return "wildcard" — regardless of the uncertainty bucket. Wildcards are never eligible as scenario axes.
2. Else if impact >= 4 AND uncertainty == "High", return "critical_uncertainty".
3. Else if impact >= 4 AND uncertainty == "Low", return "predetermined" — this signal must appear as a constant across all 4 future scenarios, never as an axis.
4. Else return "background".

Output schema:
{ bucket: "critical_uncertainty"|"predetermined"|"background"|"wildcard",
  rationale: string /* one sentence, cites impact/uncertainty values and, for wildcard, the shock mechanism */ }
```

PROMPT — ORTHOGONALITY / INDEPENDENCE CHECK

This replaces the current client-side `assessIndependence()` stub (a hashed pseudo-random "sync %") with a real grounded judgment — it must reach one of exactly three states already defined in the UI: independent, correlated, uncertain.

```
Task: Assess whether two candidate scenario axes resolve independently
of one another. Schwartz's method requires the two axes to be able to
combine into all 4 plausible quadrants – correlated axes collapse the
matrix to effectively 2 scenarios, which is a methodology violation.

Input: { axis_a: {title, body, category}, axis_b: {title, body, category},
        library_signals: [{title, category}] /* full signal library for context */ }

Decision procedure (must follow, do not skip steps):
1. If axis_a and axis_b share a common underlying causal driver (one's
   outcome mechanically constrains the other's), return "correlated"
   and name the shared driver in rationale.
2. Else if the library contains fewer than 2 supporting signals for
   either axis's category, return "uncertain" – there isn't enough
   evidence in this project to judge independence either way. Do not
   guess in this case.
3. Else return "independent".
Every rationale bullet must reference axis_a/axis_b's actual content or
a named library signal – never a generic statement like "these seem
different."

Output schema:
{ state: "independent"|"correlated"|"uncertain",
  rationale: string[] /* 1-3 bullets, each grounded */ }
```

8. Step 5 — Scenario logics (build-scenarios-modal.jsx, reaxis-modal.jsx)

Schwartz: with 2 axes fixed, define the internal logic of each of the 4 quadrants — what makes that combination a coherent, plausible world (not just a label).

ENDPOINT	TRIGGER
<code>POST</code> /projects/:id/scenarios/build	"Build Scenario Matrix" confirm (first time)
<code>POST</code> /projects/:id/scenarios/reaxis-preview	Re-axis modal step 2, before committing migration

PROMPT

```
Task: Generate the 4 scenario logics for a 2x2 built from axis_a
(vertical) and axis_b (horizontal). A "logic" is the causal argument for
why that combination of pole outcomes is an internally consistent world
– not just a mashup of two adjectives.

Input: { focal_question, horizon, axis_a: {label, pole_pos, pole_neg},
        axis_b: {label, pole_pos, pole_neg},
        predetermined_signals: [...] /* pulled from matrix_dots WHERE bucket='predetermined'
*/,
        wildcard_signals: [...] /* pulled from matrix_dots WHERE bucket='wildcard' – surfaced
to the
        model as optional shock disruptors it MAY reference in a narrative but must never
fold into
        the axis logic itself */,
        name_pool: string[] /* curated evocative names, model must pick
from or closely riff on this pool – never invent unrelated
proper nouns */ }

Rules:
- Produce exactly 4 scenarios, one per quadrant (TL/TR/BL/BR), each
  combining one pole of axis_a with one pole of axis_b.
- `logic` must explain the causal mechanism connecting the two poles
  into one coherent world in 1–2 sentences – if a quadrant combination
  is internally contradictory (e.g. both poles require the same
  precondition to differ), set `plausible:false` and explain why in
  `implausibility_note` instead of forcing a narrative.
- Weave in predetermined_signals where relevant – these hold true in
  ALL 4 quadrants and must not contradict any scenario's logic.
- `tagline` <= 6 words, `summary` <= 2 sentences, no invented statistics.

Output schema:
{ scenarios: [{ quadrant: "TL"|"TR"|"BL"|"BR", name, tagline, summary,
                logic, plausible: boolean, implausibility_note?: string }] }
```

9. Step 6 — Narratives & storyline (page-narrative.jsx, page-storyline.jsx)

Schwartz: flesh out each scenario into a full narrative — the causal chain of events from today's precursors to the realized future. Scenaric implements this as prose (Narrative page) and a 5-phase causal graph (Storyline page); both must derive from the same chain, not diverge.

ENDPOINT	TRIGGER
POST /scenarios/:id/storyline/auto-suggest	"Auto-suggest chain from AI", storyline empty state
POST /scenarios/:id/narrative/expand	"Expand with AI" on narrative page
POST /scenarios/:id/storyline/find-signal	"Find signals" gap-filler in side panel

PROMPT — STORYLINE CAUSAL CHAIN (SOURCE OF TRUTH)

Task: Build the causal chain for ONE scenario across Schwartz's 5 phases:
 Precursors (conditions already true today) → Catalysts (near-term triggering events) → First-order effects → Second-order effects →
 Scenario realized (the end state at the horizon).

Input: { scenario: {name, logic, quadrant}, axis_a, axis_b,
 library_signals: [...], predetermined_signals: [...],
 horizon: string }

Rules:

- Every node must belong to exactly one phase and connect via ≥ 1 edge (except phase-1 precursors, which may be roots).
- No forward-skipping shortcuts that assert the end state as a precursor – respect causal ordering; a node in phase N can only causally justify nodes in phase N or later (never earlier – that is the "backward in time" error state the product already flags).
- Every node's `title`+`body` must trace to a library_signal or predetermined_signal id in `grounded\_in`, OR be marked `inference:true` if it's a plausible near-term consequence the model is inferring rather than one already present in the library – cap inferred nodes at 40% of the chain.
- Edge `relationship` in {"Leads to","Enables","Amplifies","Blocks"}, `confidence` in {"Strong","Moderate","Weak"} – set confidence Weak whenever grounded_in is empty (i.e. purely inferred).
- 7-10 nodes total, at least 1 per phase.

Output schema:

```
{ sufficient_evidence: boolean,
  nodes: [{ id, phase, category(STEEP), title, body, year,
            grounded_in: string[], inference: boolean }],
  edges: [{ from, to, relationship, confidence }] }
```

PROMPT — NARRATIVE PROSE EXPANSION

```
Task: Turn an approved storyline chain (nodes+edges) into the scenario's
readable narrative paragraph, in past-tense retrospective voice ("by
{horizon_end}, X had happened...") per Schwartz's convention of writing
scenarios as history remembered from the future.

Input: { scenario: {name, tagline, summary}, storyline_nodes: [...ordered
      by phase], storyline_edges: [...]}

Rules:
- The narrative must mention every node's core fact in causal order –
  do not invent connective events not present in the chain.
- Do not introduce any named company, person, statistic, or policy not
  present in a node's title/body.
- 150-300 words. No bullet points – flowing prose.
- If storyline_nodes is empty or has <4 nodes, return
  sufficient_evidence:false instead of writing a thin narrative from
  scratch.

Output schema:
{ sufficient_evidence: boolean, narrative: string }
```

10. Step 7 — Implications (page-narrative.jsx)

Schwartz: for each scenario, work out what it would mean for the actual decision — this is where scenarios connect back to the focal question.

ENDPOINT	TRIGGER
POST /scenarios/:id/implications/generate	Narrative page load / regenerate action

PROMPT

```
Task: Derive strategic implications of ONE scenario for the focal
question – concrete "if this future happens, we would need to..."
statements, not restated scenario facts.

Input: { focal_question, scenario: {name, narrative}, categories:
        ["capital allocation","hiring/org","technology/product",
        "partnerships","other"] }

Rules:
- Each implication must name a decision or resource shift the
  organization in focal_question would plausibly need to make – derived
  from a specific fact in scenario.narrative, cited in `grounded_in_text`
  (a short quote/paraphrase of the triggering sentence).
- No generic implications that would apply to any scenario ("invest in
  talent") – must be scenario-specific.
- 3-5 implications, each 1 sentence, each tagged with one category.

Output schema:
{ implications: [{ text, category, grounded_in_text }] }
```

11. Step 8 — Indicators / signposts (page-monitoring-settings.jsx)

Schwartz: leading indicators are observable, near-term events that reveal which scenario is actually unfolding — they must be specific and checkable, not vague trends.

ENDPOINT	TRIGGER
POST /projects/:id/indicators/generate	"Add indicator" → AI suggest, or run once per scenario after narrative approval
PUT /indicators/:id/status	Manual or news-feed-triggered status update (Watch/Alert/On track)

PROMPT — GENERATE SIGNPOSTS

Task: Generate leading indicators (signposts) for ONE scenario – concrete, externally observable events that would tell an analyst this scenario is the one unfolding, distinguishable from the OTHER 3 scenarios.

Input: { scenario: {name, storyline_nodes}, sibling_scenarios: [{name, tagline}] /* the other 3, for distinguishing power */ }

Rules:

- Each indicator must be phrased as a checkable, dated, or thresholded event ("X ruling published", "Y index crosses Z%") – never a vague directional claim ("regulation increases").
- Each indicator must map to exactly one storyline_node's mechanism (`grounded\_in`) – do not invent indicators disconnected from the approved chain.
- Reject (do not output) any candidate indicator that would equally well signal one of sibling_scenarios – signposts must discriminate.
- 4-6 indicators per scenario.

Output schema:

```
{ indicators: [{ name, grounded_in: string (node id),
                  discriminates_from: string[] (sibling scenario ids) }] }
```

PROMPT — STATUS CLASSIFICATION FROM A NEWS ITEM (USED BY THE NEWS FEED → "+ ADD TO SIGNALS"/MONITORING LOOP)

Task: Given one incoming news item and the project's existing indicators, decide whether it moves any indicator's status.

Input: { news_item: {title, source, body}, indicators: [{id, name, current_status}] }

Rules:

- Only change status for indicators the news_item DIRECTLY concerns – match on named entities/mechanisms, not thematic vibes.
- Escalate to "Alert" only if the news_item reports the indicator's triggering condition has occurred or is imminent (<1 quarter). "Watch" if directionally relevant but not yet triggering. Otherwise leave unchanged.
- If uncertain whether the news_item concerns an indicator at all, leave it unchanged – do not force a match.

Output schema:

```
{ updates: [{ indicator_id, new_status, reason: string }] }
```

12. Strategic options (product extension, not in Schwartz) (page-strategy.jsx)

Wind-tunnelling: test candidate strategies against all 4 scenarios and score robustness. This is standard scenario-planning practice (Shell/GBN) that Schwartz's book gestures at but doesn't formalize as a named step — treat its grounding rules as strict as the book steps.

ENDPOINT	TRIGGER
<code>POST</code> /projects/:id/strategies/generate	"Generate options" button
<code>POST</code> /projects/:id/strategies/wind-tunnel	Re-score existing strategies after scenarios change

PROMPT

```
Task: Propose strategic options for the focal question and score each
against all 4 scenarios ("wind-tunnelling" – does this option still work
if this future happens?).

Input: { focal_question, scenarios: [{id, name, narrative, implications}] }

Rules:
- Each strategy must be actionable (a real operating/investment choice),
  not a restated goal ("succeed in all markets").
- `robust_in` may only include a scenario id if the strategy's mechanism
  is explicitly supported by that scenario's narrative or implications –
  cite the supporting text in `robust_because`. If a strategy is robust
  in 0–1 scenarios, still include it if it's the best available option,
  but flag `single_scenario_bet:true`.
- risk/cost are relative within THIS option set only (Low/Medium/High),
  derived from execution complexity and capital intensity described in
  narratives – not a generic guess.
- 3–5 strategies. Always include one full-coverage or near-coverage
  hedge if the scenario set allows one; never fabricate one that isn't
  supported.

Output schema:
{ strategies: [{ name, notes, risk: "Low"|"Medium"|"High",
                  cost: "Low"|"Medium"|"High",
                  robust_in: [{scenario_id, robust_because}],
                  single_scenario_bet: boolean }] }
```

13. Cross-cutting: Ask AI chat & project progress

Ask AI (ask-ai.jsx)

Replace the canned-reply array with a single grounded chat endpoint. It must answer only from the active project's stored data (signals, scenarios, storyline, indicators) — not general knowledge about the user's industry.

Task: Answer the user's question about their scenario-planning project.

Input: { question, project_snapshot: { focal_question, signals, axes, scenarios, storyline_summaries, indicators, strategies } }

Rules:

- Answer ONLY using project_snapshot. If the question requires info not present (e.g. asks about a market not in any signal), say so plainly and suggest which step would supply it (e.g. "add a signal about X in the Signals Library") — never answer from general knowledge as if it were project fact.
- When citing a scenario or signal, name it exactly as stored.
- Keep responses under ~120 words unless the user asks for depth.

Output schema: { answer: string, cites: string[] (ids referenced) }

Project progress (stepsComplete)

Currently client-set in the prototype. Make it server-derived so a project's tile state can never be spoofed or drift from real data:

```
steps_complete = count of gates passed, where:
  1 (Focal question)   → project.refined_focal_question IS NOT NULL
  2 (Key forces)        → count(insights) >= 1
  3 (Driving forces)    → count(signals) >= 4 (min. viable STEEP spread)
  4 (Rank forces)       → all signals have bucket assigned AND axes row exists with
independence_state != 'uncertain'
  5 (Scenario logics)   → count(scenarios WHERE plausible) == 4
  6 (Narratives)        → count(scenarios WHERE narrative IS NOT NULL) == 4
  7 (Implications)      → count(implications) >= 4*3
  8 (Indicators)        → count(indicators) >= 4*3
  9 (Strategy)          → count(strategies) >= 3
```

14. Build order

1. Schema + multi-project CRUD (projects, sources, signals, scenarios, etc.) with project-scoped auth on every route.
2. Server-derived `steps_complete` gate (§13) — wires the existing dashboard/projects UI to real state immediately, even before AI exists.
3. Global prompt scaffold + structured-output client wrapper (schema validation, retry-once, `ai_runs` audit log) — build once, reuse for every step below.
4. Step 1 (focal refine) + Step 2/3 (insight + signal extraction) — unlocks Knowledge Base and Signals Library end to end.
5. Step 4 (scoring + independence check) — Matrix page fully live.
6. Step 5 (scenario logics) for both first-build and re-axis flows.
7. Step 6 (storyline chain + narrative prose) — build storyline first since narrative prose depends on it.
8. Step 7 + 8 (implications, indicators) — both are cheap, single-call generations once scenarios/storyline exist.
9. Strategic options (§12) + Ask AI (§13) — depend on everything above being populated.
10. News-feed → indicator status classifier (§11) as a background job/webhook once indicators exist.